
Protocol Audit Report: PasswordStore

Nirenix

May 21, 2026

Protocol Summary

PasswordStore is a smart contract designed to allow a single user to store and retrieve a password on-chain. The contract is intended to ensure that only the owner can set or read the stored password.

Disclaimer

The auditor (Nirenix) makes all efforts to find as many vulnerabilities in the code in the given time period, but holds no responsibility for the findings provided in this document. A security audit by the auditor is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely focused on the security aspects of the Solidity implementation of the contracts.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High / Medium	Medium
Likelihood: Medium	High / Medium	Medium	Medium / Low
Likelihood: Low	Medium	Medium / Low	Low

Audit Details

Scope

1	./src/
2	└─ PasswordStore.sol

Roles

- **Owner:** The user who can set and read the password.
- **Outsiders:** No other party should be able to set or read the password.

Executive Summary

Issues Found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Total	3

[illegible]

```
1 myPassword
```

Recommended Mitigation:

Additionally, the `getPassword` view function should be removed, since invoking it on-chain would require sending the decryption key through a transaction —exposing it publicly through transaction calldata.

[H-2] PasswordStore::setPassword has no access control, allowing any non-owner to change the stored password**Description:**

The `setPassword` function is declared `external` and contains no access control logic. Although the contract's NatSpec describes it as a function for the owner to set a new password, the implementation does not verify that `msg.sender == s_owner`.

```
1 function setPassword(string memory newPassword) external {
2     // @audit No access control — any caller can overwrite the
      password
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact:

Any external account can call `setPassword` and overwrite the owner's password, severely breaking the contract's intended functionality.

Proof of Concept:

Add the following fuzz test to `PasswordStore.t.sol`:

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     // Exclude the legitimate owner from the fuzz input
3     vm.assume(randomAddress != owner);
4
5     // A non-owner sets a new password
6     vm.startPrank(randomAddress);
7     string memory attackPassword = "attacker_win";
8     passwordStore.setPassword(attackPassword);
9     vm.stopPrank();
10
11     // The owner reads the password back and observes it has been
      overwritten
12     vm.startPrank(owner);
13     string memory actualPassword = passwordStore.getPassword();
14     assertEq(actualPassword, attackPassword);
15 }
```

Running this test confirms that any address can successfully overwrite the password.

Recommended Mitigation:

Add an ownership check at the start of `setPassword`:

```
1 function setPassword(string memory newPassword) external {
2     if (msg.sender != s_owner) {
```

```
3         revert PasswordStore__NotOwner();
4     }
5     s_password = newPassword;
6     emit SetNetPassword();
7 }
```

Informational

[I-1] The PasswordStore::getPassword NatSpec references a parameter that does not exist

Description:

The NatSpec documentation for `getPassword` declares a `@param newPassword` tag, but the function takes no arguments:

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3   * @param newPassword The new password to set.
4   */
5  function getPassword() external view returns (string memory) {
```

The function signature is `getPassword()`, while the NatSpec describes `getPassword(string)`.

Impact:

The NatSpec documentation is misleading. This does not affect runtime behavior, but it reduces code clarity and may confuse integrators or future maintainers.

Recommended Mitigation:

Remove the incorrect `@param` line:

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3   - * @param newPassword The new password to set.
4   */
5  function getPassword() external view returns (string memory) {
```